

PRACTICAL: 03

Aim: To implement the max-heap sorting algorithm.

Algorithm:

Heap Sort
Algorithm:
Input: An unsorted array A of n elements.
Output: A sorted array A in ascending order.

1. Start
2. Read array A and n.
3. Build a MaxHeap from the input array - rearrange array so every parent node is greater than its child nodes.
4. for $i = n/2 - 1$ down to 0
 - Heapify(A, n, i)
5. In Heapify, compare the root with its left and right children, and find the largest among the three.
 - largest = i
 - left = 2*i + 1
 - right = 2*i + 2
 - if A[left] > A[largest]: largest = left
 - if A[right] > A[largest]: largest = right
6. If the largest is not the root, swap root with the largest child.
 - if largest != i
 - swap(A[i], A[largest])
7. Recursively apply heapify on the affected sub-tree, since swapping may break the heap property below.
 - Heapify(A, n, largest)
8. Repeat steps 3-5 until the entire array satisfies the Max-Heap property (Build MaxHeap) Complete.

9. Swap the root (maximum element, A[0]) with the last element of the heap.
10. Swap(A[0], A[n-1])
11. Reduce the heap size by 1 (excluding the last element, which is now in its correct sorted position).
12. Call heapify on the root to restore the Max Heap property for the reduced heap.
13. End

Program / Code:

```
#include <iostream>

using namespace std;

class MaxHeap {
public:
    int arr[100], n;

    void arrayInput() {
        cout << "Enter number of elements: ";
        cin >> n;
    }
};
```



```
    cout << "Enter the elements: ";

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}

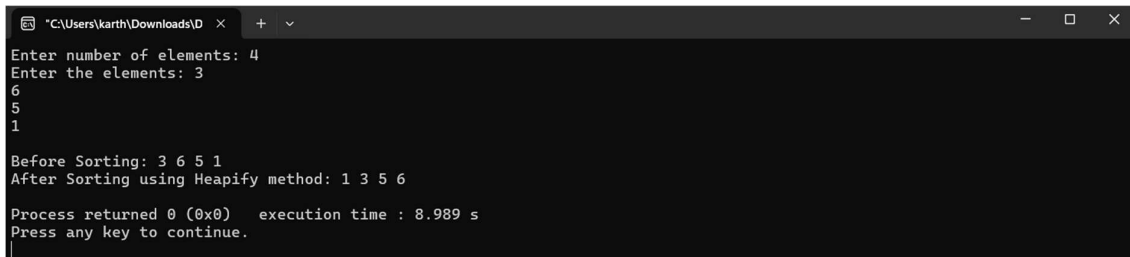
void printArray() {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void maxheapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        maxheapify(arr, n, largest);
    }
}

void heapsort() {
    for (int i = n / 2 - 1; i >= 0; i--) {
        maxheapify(arr, n, i);
    }
}
```

```
    }  
    for (int i = n - 1; i > 0; i--) {  
        swap(arr[0], arr[i]);  
        maxheapify(arr, i, 0);  
    }  
}  
};  
  
int main() {  
    MaxHeap mh;  
    mh.arrayInput();  
    cout << "\nBefore Sorting: ";  
    mh.printArray();  
    mh.heapsort();  
    cout << "After Sorting using Heapify method: ";  
    mh.printArray();  
    return 0;  
}
```

Output:



```
"C:\Users\karth\Downloads\D" X + v  
Enter number of elements: 4  
Enter the elements: 3  
6  
5  
1  
  
Before Sorting: 3 6 5 1  
After Sorting using Heapify method: 1 3 5 6  
  
Process returned 0 (0x0) execution time : 8.989 s  
Press any key to continue.
```

Time Complexity:

Time Complexity:

MaxHeapify(arr, size, i) { $\rightarrow T(n)$

int length = size - 1;

int l = (2*i + 1);

while (l < size & arr[l] > arr[largest]) {

largest = l;

if (size < 2) {

largest = i;

if (arr[largest] < arr[i]) {

swap(arr[largest], arr[i]);

maxHeapify(arr, size, i); $\rightarrow T(n/2)$

}

}

heapSort (size, i, i-1) { $\rightarrow n$

maxHeapify(arr, size-i); $\rightarrow \log n$

}

for (i = size; i > 1; i--) { $\rightarrow n-1$

swap(arr[i], arr[1]); $\rightarrow 1$

maxHeapify(arr, size, 1); $\rightarrow \log n$

}

for maxHeapify

$T(n) = T(n/2) + 1$

using master method

$T(n) = 1$

$a=1, b=2, k=0, p=0$

$\log_b a = \log_2 1 = \frac{\log 1}{\log 2} = 0 = k$

$p > -1 \Rightarrow 0 > -1$

$T(n) = O(n^{\log_2 1})$

$O(n^{\log_2 1})$

$O(n^0 \log n)$

$T(n) = O(\log n)$

Build max Heap

$T(n) = \frac{n}{2} \times \log n$

$+ O(n \log n)$

heapSort

$T(n) = (n-1) \times \log n$

$= O(n \log n)$

Overall heap sort:

$T(n) = O(n \log n) + O(n \log n)$

$= O(n \log n)$

Best case $T(n) = O(n \log n)$

Worst case:

$T(n) = T(n/2) + O(1)$

$= O(\log n)$

for loop $n-1$ times

$T(n) = O(n) + (n-1) \times O(\log n)$

$T(n) = O(n \log n)$ worst case

Average case:

Same like worst case

$T(n) = O(n \log n)$

Average case

Conclusion:

In this practical, we implemented the Max-Heap Sort algorithm successfully using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is $O(n \log n)$.